

Architecture of Linux

Let's first start with the basic knowledge of the Linux operating system.

Linux operating system

An operating system can be described as an interface among the computer hardware and the user of any computer. It is a group of software that handles the resources of the computer hardware and facilitates basic services for computer programs.

An operating system is an essential component of system software within a computer system. The primary aim of an operating system is to provide a platform where a user can run any program conveniently or efficiently.

On the other hand, [Linux](#) OS is one of the famous versions of the UNIX OS. It is developed to provide a low-cost or free OS for several personal computer system users. Remarkably, it is a complete OS including an **X Window System**, **Emacs editor**, [IP/TCP](#), **GUI** (graphical user interface), etc.

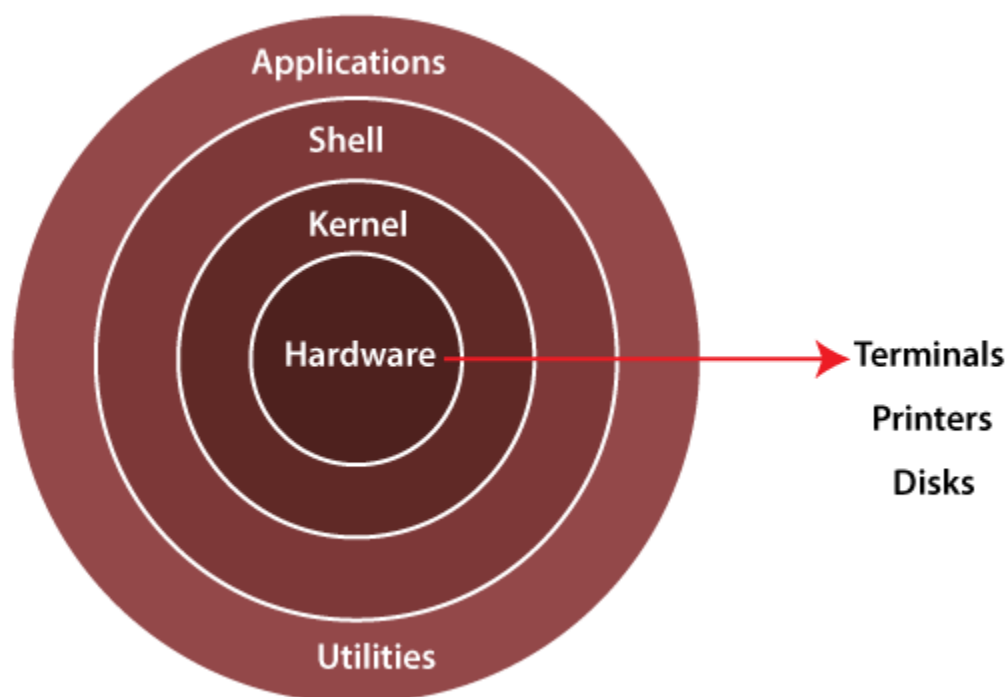
Linux operating system history

In 1991, the Linux history started with the starting of a particular project by the Finland student **Linus Torvalds** for creating a new free **OS kernel**. The final Linux Kernel was remarked by continuous development throughout the history since then.

- Linux was proposed by the Finland student Linus Torvalds in 1991.
- HP-UX (**Hewlett Packard**) 8.0 version was published.
- Hewlett Packard 9.0 version was published in 1992.
- FreeBSD 1.0 version and **NetBSD** 8 version was released in 1993.
- Red Hat Linux was proposed in 1994. Caldera was detected by Ransom love and Bryan Sparks and NetBSD 1.0 version published.
- HP-UX 10.0 version and FreeBSD 2.0 version was released in 1995.
- K Desktop Environment was established by **Matthias Ettrich** in 1996.
- HP-UX 11.0 version was released in 1997.
- The IRIX 6.5 version, i.e., the fifth SGI UNIX generation, Free BSD 3.0 version, and Sun Solaris 7 OS was released in 1998.
- The **Caldera System** agreement with professional services division and SCO server software division was released in 2000.
- **Linus Torvalds** published the Linux version 2.4 source code in 2001.
- **Microsoft** filed the Trademark collection against Lindows.com in 2001.
- Lindows name was modified to Linspire in 2004.
- The first publication of **Ubuntu** was published in 2004.

- The openSUSE project started a free distribution from the community of **Novell** In 2005.
 - **Oracle** published its Red Hat distribution in 2006.
 - **Dell** begun laptop distribution with Ubuntu which was pre-installed on it in 2007.
 - Linux kernel version 3.0 was released in 2011.
 - Linux-based android of Google insisted 75% of the market share of the Smartphone, based on the number of phones exported in 2013.
 - Ubuntu insisted on 20000000+ users in 2014.
-

Architecture of Linux system



The Linux operating system's architecture mainly contains some of the components: **the Kernel, System Library, Hardware layer, System, and Shell utility.**

1. Kernel:- The kernel is one of the core section of an operating system. It is responsible for each of the major actions of the Linux OS. This operating system contains distinct types of modules and cooperates with underlying hardware directly. The kernel facilitates required abstraction for hiding details of low-level hardware or application programs to the system. There are some of the important kernel types which are mentioned below:

- Monolithic Kernel
 - Micro kernels
 - Exo kernels
 - Hybrid kernels
-

2. System Libraries:- These libraries can be specified as some special functions. These are applied for implementing the operating system's functionality and don't need code access rights of the modules of kernel.

3. System Utility Programs:- It is responsible for doing specialized level and individual activities.

4. Hardware layer:- Linux operating system contains a hardware layer that consists of several peripheral devices like [CPU](#), [HDD](#), and [RAM](#).

5. Shell:- It is an interface among the kernel and user. It can afford the services of kernel. It can take commands through the user and runs the functions of the kernel. The shell is available in distinct types of OSes. These operating systems are categorized into two different types, which are the **graphical shells** and **command-line shells**.

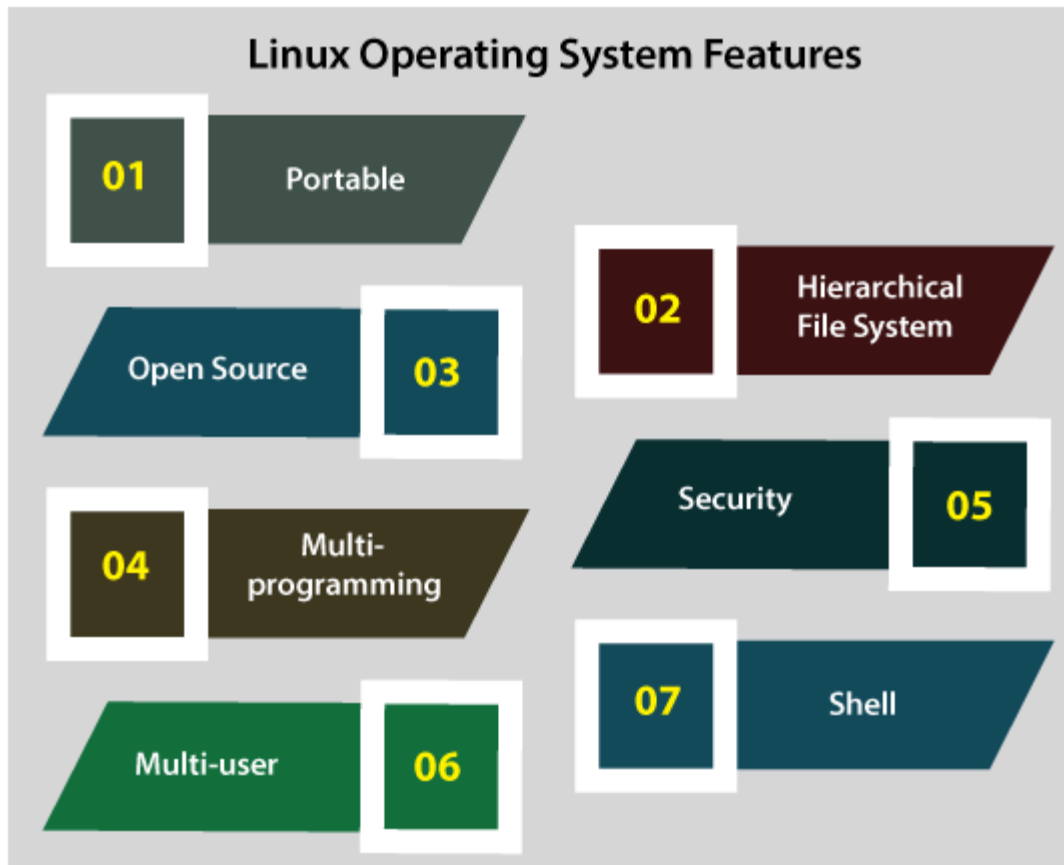
The graphical line shells facilitate the graphical user interface, while the command line shells facilitate the command line interface. Thus, both of these shells implement operations. However, the graphical user interface shells work slower as compared to the command-line interface shells.

There are a few types of these shells which are categorized as follows:

- Korn shell
 - Bourne shell
 - C shell
 - POSIX shell
-

Linux Operating System Features

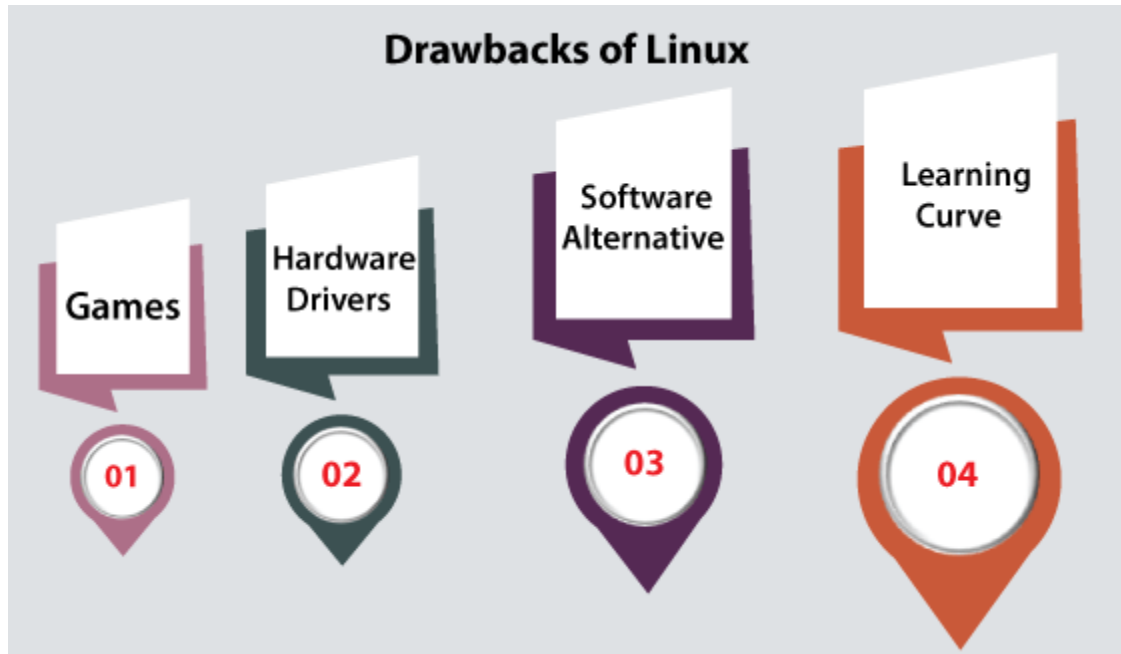
Some of the primary features of Linux OS are as follows:



- **Portable:** Linux OS can perform different types of hardware and the kernel of Linux supports the installation of any type of hardware environment.
- **Open source:** Linux operating system source code is available freely and for enhancing the capability of the Linux OS, several teams are performing in collaboration.
- **Multiprogramming:** Linux OS can be defined as a multiprogramming system. It means more than one application can be executed at the same time.
- **Multi-user:** Linux OS can also be defined as a multi-user system. It means more than one user can use the resources of the system such as **application programs, memory, or RAM** at the same time.
- **Hierarchical file system:** Linux OS affords a typical file structure where user files or system files are arranged.
- **Security:** Linux OS facilitates user security systems with the help of various features of authentication such as controlled access to specific files, password protection, or data encryption.
- **Shell:** Linux operating system facilitates a unique interpreter program. This type of program can be applied for executing commands of the operating system. It

can be applied to perform various types of tasks such as call application programs and others.

Drawbacks of Linux



- **Hardware drivers:** Most of the users of Linux face an issue while using Linux. Various companies of hardware prefer to build drivers for Mac or Windows due to they contain several users than Linux. Linux has small drivers for peripheral hardware than windows.
 - **Software alternative:** Let's take the Photoshop example which is a famous tool for graphic editing. Photoshop exists for Windows; however, it is not available in Linux. Also, there are some other tools for photo editing but the Photoshop tool is more powerful as compare to others. Another example is MS office which is not present for Linux users.
 - **Learning curve:** Linux isn't a very user-friendly operating system. Hence, it might be confusing for many beginners. Getting begun with Windows is efficient and easy for many beginners; however, understanding Linux working is complex. We have to understand the command line interface and finding for newer software is a little bit complex as well. When we face any issue in the OS, the searching solution is very problematic. Also, there are various experts for Mac and Windows as compare to Linux.
 - **Games:** Several games are developed for Windows but unfortunately not for Linux. Because the platform of Windows is used widely. So, the developers of the games are more interested in windows.
-

Linux Operating System Applications

Linux is a billion-dollar corporation nowadays. Thousands of governments and companies are using Linux operating system across the world because of lower money, time, licensing fee, and affordability. Linux can be used within several types of electronic devices. These electronic devices are easily available for users worldwide. A few of the famous Linux-based electronic devices are listed below:

- Yamaha Motive Keyboard
 - Volvo In-Car Navigation System
 - TiVo Digital Video Recorder
 - Sony Reader
 - Sony Bravia Television
 - One Laptop Per child XO2
 - Motorola MotoRokr EM35 phone
 - Lenovo IdeaPad S9
 - HP Mini 1000
 - Google Android Dev Phone 1
 - Garmin Nuvi 860, 880, and 5000
 - Dell Inspiron Mini 9 and 12
-

Linux Distribution

It is an OS that is composed of a software-based collection on Linux kernel or we can say the distribution includes the Linux Kernel. It is supporting software and libraries. We can obtain Linux-based OS by downloading any Linux distribution. These types of distributions exists for distinct types of devices such as **personal computers, embedded devices**, etc. Around more than 600 Linux distributions are existed and a few of the famous Linux distributions are listed as follows:

- Deepin
 - OpenSUSE
 - Fedora
 - Solus
 - Debian
 - Ubuntu
 - Elementary
 - Linux Mint
 - Manjaro
 - MX Linux
-

Are Ubuntu and Linux Differ?

YES.

The primary difference between window and Linux is that window is open source and free OS and its Linux distribution based on Debian, Whereas Linux is a large collection of open-source OSes that are working based on Linux kernel.

Besides, Ubuntu is a distribution of Linux and Linux is a core system. Ubuntu is integrated by Canonical Ltd. and published in 2004 and Linux is integrated by Linus Torvalds and published in 1991.

User mode vs Kernel mode

The code of kernel component runs in a unique privilege mood known as **kernel mode** along with complete access to every computer resource. This code illustrates an individual process, runs in an individual address space, and don't need the context switch. Hence, it is very fast and efficient.

Kernel executes all the processes and facilitates various services of a system to the processes. Also, it facilitates secured access to processes to hardware.

The support code that is not needed to execute in kernel mode is inside the system library. The user programs and other types of system programs are implemented in the user mode.

It includes no access to kernel mode and system hardware. User utilities/programs use the system libraries for accessing kernel functions to obtain low-level tasks of the system.

Internal and External Commands in Linux

1. **Introduction to Linux Commands** Linux commands are used to interact with the operating system via the terminal. These commands are categorized into:
 - Internal Commands (Built-in Commands)
 - External Commands
2. **Internal Commands Definition:** Internal commands are built into the shell (e.g., Bash). These commands are executed directly by the shell.
Features:
 - Stored in the shell's memory.
 - Do not require separate executable files.

- Faster execution compared to external commands.

Ex : \$type cd

cd is a shell builtin

//specifying that cd is
internal type//

Examples of Internal Commands:

- cd: Change directory
- pwd: Print current working directory
- echo: Display a message or variable value
- set: Set shell variables
- exit: Close the terminal session
- alias: Create shortcuts for commands
- history: Show command history

3. External Commands Definition: External commands are stored as separate executable files in directories like /bin, /usr/bin, and /sbin.

Features:

- Require an executable file.
- Can be customized or replaced.
- Slower execution compared to internal commands.

Ex : \$type cat

cat is /bin/cat

//specifying that cat is
external type//

Examples of External Commands:

- ls: List files and directories
- grep: Search for a pattern in files
- find: Search for files in a directory
- tar: Archive files
- cp: Copy files and directories
- chmod: Change file permissions

4. How to Identify Internal and External Commands

- Use the "type" command: `type cd`, `type ls`
- Use the "which" command for external commands: `which ls`

- Check the "help" command for internal commands: `help cd`

5. Difference Between Internal and External Commands

Feature	Internal Commands	External Commands
Location	Built into the shell	Stored in system directories
Execution Speed	Faster	Slower
Dependency	No external files required	Needs separate executable files
Example Commands	cd, pwd, echo	ls, grep, cp

6. Conclusion

Understanding internal and external commands is essential for efficient Linux usage. Internal commands are built into the shell, whereas external commands are stored separately and executed when required.

Directory Commands

directory-related commands are essential for navigating and managing the file system. Here are some commonly used directory commands:

1. pwd (Print Working Directory)

- Description: Displays the current directory you are in.
- Syntax: pwd
- Example: \$ pwd
/home/username

2. cd (Change Directory)

- Description: Changes the current directory.
- Syntax: cd [directory]
- Examples:
 - Move to a specific directory:
\$ cd /home/username/Documents

3. ls (List Directory Contents)

- Description: Lists files and directories in the current directory.
- Syntax: ls [options] [directory]
- Common Options:
 - -l: Long listing format.

- -a: Show hidden files (files starting with .).
 - -h: Human-readable file sizes.
- Examples: List files in the current directory:
 - \$ ls
- List files in a specific directory:
 - \$ ls /home/username/Documents
 - List files with details:
 - \$ ls -l

4. mkdir (Make Directory)

- Description: Creates a new directory.
- Syntax: mkdir [options] [directory_name]
- Common Options:
 - -p: Create parent directories as needed.
- Examples:
 - Create a single directory:
 - \$ mkdir new_directory
 - Create nested directories:
 - \$ mkdir -p parent/child/grandchild

5. rmdir (Remove Directory)

- Description: Removes an empty directory.
- Syntax: rmdir [directory_name]
- Example:
 - \$ rmdir empty_directory

6. rm (Remove)

- Description: Removes files or directories.
- Syntax: rm [options] [file/directory]
- Common Options:
 - -r: Recursively remove directories and their contents.
 - -f: Force removal without prompting.
- Examples:
 - Remove a file:
 - \$ rm file.txt
 - Remove a directory and its contents:
 - \$ rm -r directory_name

7. cp (Copy)

- Description: Copies files or directories.
- Syntax: cp [options] [source] [destination]
- Common Options:
 - -r: Recursively copy directories.

- Examples:
 - Copy a file:
\$ cp file.txt /home/username/Documents/
 - Copy a directory:
\$ cp -r directory_name /home/username/Documents/

8. mv (Move)

- Description: Moves or renames files or directories.
- Syntax: mv [options] [source] [destination]
- Examples:
 - Move a file:
\$ mv file.txt /home/username/Documents/
 - Rename a file:
\$ mv old_name.txt new_name.txt

9. find (Find Files/Directories)

- Description: Searches for files or directories based on specified criteria.
- Syntax: find [path] [expression]
- Examples:
 - Find a file by name:
\$ find /home/username -name "file.txt"
 - Find directories:
\$ find /home/username -type d

10. tree (Display Directory Structure)

- Description: Displays the directory structure in a tree-like format.
- Syntax: tree [options] [directory]
- Example: \$ tree /home/username

11. du (Disk Usage)

- Description: Estimates file and directory space usage.
- Syntax: du [options] [directory]
- Common Options:
 - -h: Human-readable format.
 - -s: Summary of total size.
- Example: \$ du -sh /home/username

12. ln (Create Links)

- Description: Creates hard or symbolic links to files or directories.
- Syntax: ln [options] [target] [link_name]
- Common Options:
 - -s: Create a symbolic (soft) link.
- Examples:

- Create a hard link:
\$ ln file.txt link_to_file.txt
- Create a symbolic link:
\$ ln -s /path/to/file.txt symlink_to_file.txt

13. chmod (Change Mode)

- Description: Changes the permissions of a file or directory.
- Syntax: chmod [options] [mode] [file/directory]
- Example: \$ chmod 755 script.sh

14. chown (Change Owner)

- Description: Changes the owner and group of a file or directory.
- Syntax: chown [options] [owner][:group] [file/directory]
- Example: \$ chown username:groupname file.txt

File and Disk Related Commands

1. pwd (Print Working Directory)

- Description: Displays the current directory you are in.
- Syntax: pwd
- Example: \$ pwd
/home/username

2. cd (Change Directory)

- Description: Changes the current directory.
- Syntax: cd [directory]
- Examples:
 - Move to a specific directory:
\$ cd /home/username/Documents
 - Move to the home directory:
\$ cd ~
 - Move up one directory level:
\$ cd ..

3. ls (List Directory Contents)

- Description: Lists files and directories in the current directory.
- Syntax: ls [options] [directory]
- Common Options:
 - -l: Long listing format.
 - -a: Show hidden files (files starting with .).
 - -h: Human-readable file sizes.
- Examples:

- List files in the current directory:
\$ ls
- List files in a specific directory:
\$ ls /home/username/Documents
- List files with details:
\$ ls -l

4. mkdir (Make Directory)

- Description: Creates a new directory.
- Syntax: mkdir [options] [directory_name]
- Common Options:
 - -p: Create parent directories as needed.
- Examples:
 - Create a single directory:
\$ mkdir new_directory
 - Create nested directories:
\$ mkdir -p parent/child/grandchild

5. rmdir (Remove Directory)

- Description: Removes an empty directory.
- Syntax: rmdir [directory_name]
- Example:
\$ rmdir empty_directory

6. rm (Remove)

- Description: Removes files or directories.
- Syntax: rm [options] [file/directory]
- Common Options:
 - -r: Recursively remove directories and their contents.
 - -f: Force removal without prompting.
- Examples:
 - Remove a file:
\$ rm file.txt
 - Remove a directory and its contents:
\$ rm -r directory_name

7. cp (Copy)

- Description: Copies files or directories.
- Syntax: cp [options] [source] [destination]
- Common Options:
 - -r: Recursively copy directories.
- Examples:
 - Copy a file:
\$ cp file.txt /home/username/Documents/

- Copy a directory:
\$ cp -r directory_name /home/username/Documents/

8. mv (Move)

- Description: Moves or renames files or directories.
- Syntax: mv [options] [source] [destination]
- Examples:
 - Move a file:
\$ mv file.txt /home/username/Documents/
 - Rename a file:
\$ mv old_name.txt new_name.txt

9. find (Find Files/Directories)

- Description: Searches for files or directories based on specified criteria.
- Syntax: find [path] [expression]
- Examples:
 - Find a file by name:
\$ find /home/username -name "file.txt"
 - Find directories:
\$ find /home/username -type d

10. tree (Display Directory Structure)

- Description: Displays the directory structure in a tree-like format.
- Syntax: tree [options] [directory]
- Example:
\$ tree /home/username

11. du (Disk Usage)

- Description: Estimates file and directory space usage.
- Syntax: du [options] [directory]
- Common Options:
 - -h: Human-readable format.
 - -s: Summary of total size.
- Example:
\$ du -sh /home/username

12. ln (Create Links)

- Description: Creates hard or symbolic links to files or directories.
- Syntax: ln [options] [target] [link_name]
- Common Options:
 - -s: Create a symbolic (soft) link.
- Examples:
 - Create a hard link:

- Create a symbolic link:
`$ ln -s /path/to/file.txt symlink_to_file.txt`

- Description: Changes the permissions of a file or directory.
- Syntax: `chmod [options] [mode] [file/directory]`
- Example:
\$ `chmod 755 script.sh`

- **Description:** Changes the owner and group of a file or directory.
- **Syntax:** `chown [options] [owner][:group] [file/directory]`
- **Example:**
`$ chown username:groupname file.txt`

- **Description:** Creates an empty file or updates the timestamp of an existing file.
- **Syntax:** touch [filename]
- **Examples:**
 - Create a new file:
\$ touch file.txt
 - Update the timestamp of an existing file:
\$ touch existing_file.txt

- Description: Displays the contents of a file or concatenates multiple files.
- Syntax: `cat [options] [file]`
- Common Options:
 - `-n`: Number all output lines.
 - `-b`: Number non-empty lines.
- Examples:
 - Display file content:
`$ cat file.txt`
 - Concatenate multiple files:
`$ cat file1.txt file2.txt > combined.txt`

- Description: Displays file content one page at a time (scrollable).
- Syntax: `less [filename]`
- Example:
`$ less large_file.txt`

- Navigation:
 - Use Space to scroll down, b to scroll up, and q to quit.
-

18. more (View File Content Page by Page)

- Description: Similar to less, but with fewer features.
 - Syntax: more [filename]
 - Example:
\$ more large_file.txt
-

19. head (Display Beginning of a File)

- Description: Displays the first few lines of a file (default: 10 lines).
 - Syntax: head [options] [filename]
 - Common Options:
 - -n [number]: Display the first n lines.
 - Example:
\$ head -n 5 file.txt
-

20. tail (Display End of a File)

- Description: Displays the last few lines of a file (default: 10 lines).
 - Syntax: tail [options] [filename]
 - Common Options:
 - -n [number]: Display the last n lines.
 - -f: Continuously monitor the file for new content (useful for logs).
 - Examples:
 - Display the last 5 lines:
\$ tail -n 5 file.txt
 - Monitor a log file:
\$ tail -f /var/log/syslog
-

21. cp (Copy Files)

- Description: Copies files or directories.
 - Syntax: cp [options] [source] [destination]
 - Common Options:
 - -r: Recursively copy directories.
 - -i: Prompt before overwriting.
 - Examples:
 - Copy a file:
\$ cp file.txt /home/username/Documents/
 - Copy a directory:
\$ cp -r directory_name /home/username/Documents/
-

22. mv (Move or Rename Files)

- Description: Moves or renames files or directories.

- Syntax: mv [options] [source] [destination]
 - Examples:
 - Move a file:
\$ mv file.txt /home/username/Documents/
 - Rename a file:
\$ mv old_name.txt new_name.txt
-

23. rm (Remove Files)

- Description: Deletes files or directories.
 - Syntax: rm [options] [file/directory]
 - Common Options:
 - -r: Recursively remove directories and their contents.
 - -f: Force removal without prompting.
 - Examples:
 - Delete a file:
\$ rm file.txt
 - Delete a directory and its contents:
\$ rm -r directory_name
-

24. ln (Create Links)

- Description: Creates hard or symbolic links to files.
 - Syntax: ln [options] [target] [link_name]
 - Common Options:
 - -s: Create a symbolic (soft) link.
 - Examples:
 - Create a hard link:
\$ ln file.txt link_to_file.txt
 - Create a symbolic link:
\$ ln -s /path/to/file.txt symlink_to_file.txt
-

25. file (Determine File Type)

- Description: Determines the type of a file (e.g., text, binary, directory).
 - Syntax: file [filename]
 - Example: \$ file file.txt
file.txt: ASCII text
-

26. find (Search for Files)

- Description: Searches for files or directories based on specified criteria.
- Syntax: find [path] [expression]
- Examples:
 - Find a file by name:
\$ find /home/username -name "file.txt"
 - Find files larger than 100MB:

```
$ find /home/username -size +100M
```

27. grep (Search Text in Files)

- Description: Searches for a specific pattern in files.
 - Syntax: `grep [options] [pattern] [file]`
 - Common Options:
 - `-i`: Case-insensitive search.
 - `-r`: Recursively search in directories.
 - Examples:
 - Search for a word in a file:

```
$ grep "hello" file.txt
```
 - Search recursively in a directory:

```
$ grep -r "hello" /home/username/
```
-

29. wc (Word Count)

- Description: Counts lines, words, and characters in a file.
 - Syntax: `wc [options] [file]`
 - Common Options:
 - `-l`: Count lines.
 - `-w`: Count words.
 - `-c`: Count characters.
 - Example:

```
$ wc file.txt
```
-

30. diff (Compare Files)

- Description: Compares two files line by line.
 - Syntax: `diff [file1] [file2]`
 - Example:

```
$ diff file1.txt file2.txt
```
-

31. chmod (Change File Permissions)

- Description: Changes the permissions of a file.
 - Syntax: `chmod [options] [mode] [file]`
 - Example:

```
$ chmod 755 script.sh
```
-

32. chown (Change File Ownership)

- Description: Changes the owner and group of a file.
 - Syntax: `chown [options] [owner][:group] [file]`
 - Example:

```
$ chown username:groupname file.txt
```
-

33. stat (Display File Details)

- Description: Displays detailed information about a file (e.g., size, permissions, timestamps).
 - Syntax: `stat [file]`
 - Example:
`$ stat file.txt`
-

34. tar (Archive Files)

- Description: Compresses or extracts files in a .tar archive.
 - Syntax:
 - Create: `tar -cvf [archive_name.tar] [files/directories]`
 - Extract: `tar -xvf [archive_name.tar]`
 - Examples:
 - Create a tar archive:
`$ tar -cvf archive.tar file1.txt file2.txt`
 - Extract a tar archive:
`$ tar -xvf archive.tar`
-

35. gzip / gunzip (Compress/Decompress Files)

- Description: Compresses or decompresses files using .gz format.
- Syntax:
 - Compress: `gzip [file]`
 - Decompress: `gunzip [file.gz]`
- Example: `$ gzip file.txt`
`$ gunzip file.txt.gz`

general utilities of Linux

It includes commands for copying, deleting, changing ownership, searching, reporting disk space, testing network connections, downloading files, and managing files.

Copy

* `cp`: A core GNU utility that copies files and directories

Delete

* `rm`: Deletes files and directories to keep your file system organized

Change ownership

- * **chown**: Changes the ownership of files and directories to a different user or group

Search

- * **grep**: A general-purpose tool that searches for any kind of text

Report disk space

- * **df**: Reports how much disk space is being used by a file system

Test network connections

- * **ping**: Tests network connections and troubleshoots connectivity issues

Download files

- * **wget**: Downloads files from the internet in the background

Manage files

- * **tar**: Creates archive files, also known as tarballs, to store multiple files

- * **ls**: Lists files or directories in the current directory

Linux general purpose utility command list man, who, cat, cd, cp, ps, ls, mv, rm, mkdir, rmdir, echo, more, date, time, kill, history, chmod, chown, finger, pwd, cal, logout, shutdown.
